

Especificação de Projeto — Entrega Final

FightDrill — Plataforma Web de Treinamento para Artes Marciais

Pedro Artur de Aguiar Cabral Ruy Agostinho Otoni Vieira Neto
Vitor Pedrosa Brito dos Santos

Maio de 2026

Sumário

1	Introdução	3
1.1	Distribuição dos Casos de Uso por Membro	3
2	UC02 — Executar Timing Drill (Pedro)	4
2.1	Descrição	4
2.2	Fluxo Principal	4
2.3	Fluxos Alternativos	5
2.3.1	Pausa e Retomada	5
2.3.2	Configuração Inválida	5
2.3.3	Encerramento Antecipado	5
2.4	Pré-condições	5
2.5	Pós-condições	5
2.6	Requisitos Relacionados	5
2.7	Diagrama de Atividades — UC02	6
2.7.1	Nós e Decisões	6
2.7.2	Texto Estruturado para Reprodução em EA	6
2.8	Diagrama de Sequência — UC02	7
2.8.1	Linhas de Vida	7
2.8.2	Mensagens (formato sintético)	7
2.8.3	Fragmento Combinado Obrigatório	9
2.9	Implementação	9
3	UC04 — Executar Footwork Drill (Vitor)	10
3.1	Descrição	10
3.2	Fluxo Principal	10
3.3	Fluxos Alternativos	11
3.3.1	Pausa e Retomada	11
3.3.2	Nenhuma Movimentação Seleccionada	11
3.3.3	Biblioteca Vazia	11
3.3.4	Encerramento Antecipado	11
3.4	Pré-condições	11

3.5	Pós-condições	12
3.6	Requisitos Relacionados	12
3.7	Diagrama de Atividades — UC04	12
3.7.1	Texto Estruturado	12
3.7.2	Nós de Decisão (atende ao requisito mínimo)	13
3.8	Diagrama de Sequência — UC04	13
3.8.1	Linhas de Vida	13
3.8.2	Mensagens	13
3.8.3	Fragmentos Combinados (atende ao requisito mínimo)	15
3.9	Implementação	15
4	UC05 — Gerenciar Combos (Ruy)	16
4.1	Descrição	16
4.2	Fluxo Principal — Listar e Adicionar Combo	16
4.3	Fluxos Alternativos	16
4.3.1	Nome Inválido (passo 5)	16
4.3.2	Nome Duplicado (passo 6)	17
4.3.3	Excluir Combo	17
4.3.4	Estado Vazio	17
4.4	Pré-condições	17
4.5	Pós-condições	17
4.6	Requisitos Relacionados	17
4.7	Diagrama de Atividades — UC05	18
4.7.1	Texto Estruturado	18
4.7.2	Nós de Decisão (atende ao requisito mínimo)	18
4.8	Diagrama de Sequência — UC05	19
4.8.1	Linhas de Vida	19
4.8.2	Mensagens	19
4.8.3	Fragmentos Combinados (atende ao requisito mínimo)	20
4.9	Implementação	20
5	Mapa de Implementação por Membro	21
6	Rastreabilidade UC × RF × RN × Membro	22

Capítulo 1

Introdução

Este documento corresponde à terceira entrega do projeto **FightDrill**. Diferentemente da entrega intermediária, em que cada membro do grupo especificou um caso de uso, nesta entrega cada membro selecionou **um caso de uso distinto** dos previamente apresentados, para o qual elaborou:

- um **diagrama de atividades** com, ao menos, um nodo de decisão;
- um **diagrama de sequência** com, ao menos, um fragmento combinado;
- a **implementação** correspondente em Python, em conformidade com os diagramas.

As atualizações no diagrama de classes resultantes desta modelagem serão aplicadas diretamente no arquivo `.eap` e, portanto, não são detalhadas neste documento.

1.1 Distribuição dos Casos de Uso por Membro

Membro	Caso de Uso (Entrega Intermediária)	Caso de Uso (Entrega Final)
Pedro Artur de Aguiar Cabral	UC01 — Executar Round Timer	UC02 — Executar Timing Drill
Vitor Pedrosa Brito dos Santos	UC03 — Executar Combo Drill	UC04 — Executar Footwork Drill
Ruy Agostinho Otoni Vieira Neto	UC07 — Cadastrar Treino Personalizado	UC05 — Gerenciar Combos

Capítulo 2

UC02 — Executar Timing Drill (Pedro)

2.1 Descrição

Permite ao praticante configurar e executar um treino do tipo **Timing Drill**, no qual o sistema emite estímulos sonoros e anúncios por voz em **intervalos aleatórios** dentro de uma faixa configurada, para uma técnica-alvo previamente informada. O treino encerra-se quando a duração total é atingida.

2.2 Fluxo Principal

1. O praticante acessa a tela do Timing Drill.
2. O sistema exibe o formulário com os campos: duração total, intervalo mínimo, intervalo máximo e técnica-alvo.
3. O praticante preenche os valores e aciona o início.
4. O sistema valida a configuração (RN: $\text{min} < \text{max}$, durações positivas, técnica não vazia).
5. O sistema oculta o formulário, exibe a tela de execução e inicia a contagem regressiva de 3 s (com beeps).
6. Ao término da contagem regressiva, o sistema inicia a sessão e **sorteia o próximo instante de estímulo**.
7. A cada segundo, o sistema avança o contador e verifica:
 - se a duração total foi atingida, encerra a sessão (passo 9);
 - se o instante agendado foi atingido, emite **STIMULUS** (passo 8); caso contrário, segue para o próximo segundo.
8. O sistema reproduz beep de início, anuncia a técnica-alvo por voz e exibe o nome destacado na tela; em seguida, sorteia o próximo instante.

9. Ao fim da duração total, o sistema emite sinal sonoro de encerramento, anuncia mensagem de conclusão e retorna ao formulário.

2.3 Fluxos Alternativos

2.3.1 Pausa e Retomada

- A. Durante a execução, o praticante aciona **Pausar**.
- B. O sistema marca a sessão como pausada; **tick** deixa de avançar e nenhum estímulo é emitido (RF10).
- C. Ao acionar **Retomar**, o sistema continua a contagem a partir do mesmo ponto.

2.3.2 Configuração Inválida

- 4a. Os valores informados violam alguma regra (`min` $\geq$ `max`, duração $\leq$ 0, técnica vazia).
- 4b. O sistema exibe a mensagem de erro abaixo do formulário.
- 4c. Retorna ao passo 2.

2.3.3 Encerramento Antecipado

- A. O praticante aciona **Parar**.
- B. O sistema para o `setInterval`, encerra a sessão e retorna ao formulário.

2.4 Pré-condições

Nenhuma.

2.5 Pós-condições

A sessão é concluída (ou encerrada antecipadamente) e a aplicação retorna ao estado de configuração.

2.6 Requisitos Relacionados

RF02, RF10, RF11, RN06.

2.7 Diagrama de Atividades — UC02

2.7.1 Nós e Decisões

- **Início.**
- *Ação:* Exibir formulário de configuração.
- *Ação:* Praticante informa duração, intervalos e técnica.
- *Ação:* Acionar início.
- **Decisão D1 — Configuração válida?**
 - *Não:* exibir erro → retornar a “Exibir formulário”.
 - *Sim:* prosseguir.
- *Ação:* Exibir contagem regressiva (3, 2, 1) com beeps.
- *Ação:* Iniciar sessão e sortear próximo estímulo.
- *Ação (loop de tick):* Avançar 1 segundo.
- **Decisão D2 — Duração total atingida?**
 - *Sim:* emitir sinal de encerramento → “Anunciar fim” → retornar ao formulário → **Fim**.
 - *Não:* prosseguir.
- **Decisão D3 — Sessão pausada?**
 - *Sim:* aguardar próximo tick (sem avançar contador) → voltar a “Avançar 1 segundo”.
 - *Não:* prosseguir.
- **Decisão D4 — Atingiu instante de estímulo agendado?**
 - *Sim:* “Emitir beep + anúncio de voz + destaque visual” → “Sortear próximo instante” → voltar a “Avançar 1 segundo”.
 - *Não:* voltar a “Avançar 1 segundo”.

2.7.2 Texto Estruturado para Reprodução em EA

[Start]

```
-> (Exibir formulario)
-> (Praticante preenche campos)
-> (Acionar inicio)
-> <D1: configuracao valida?>
    - nao -> (Exibir erro) -> (Exibir formulario)
    - sim ->
-> (Contagem regressiva 3..1)
```

```

-> (Iniciar sessao e sortear proximo estimulo)
-> (Avancar 1s) // ponto de retorno do loop
-> <D2: elapsed >= total_duration?>
    - sim -> (Emitir end signal) -> (Anunciar fim) -> (Voltar ao formulario) -> [E
    - nao ->
-> <D3: pausada?>
    - sim -> (Aguardar) -> (Avancar 1s)
    - nao ->
-> <D4: elapsed >= next_stimulus_at?>
    - sim -> (Emitir STIMULUS: beep + anuncio + destaque)
        -> (Sortear proximo estimulo)
        -> (Avancar 1s)
    - nao -> (Avancar 1s)

```

2.8 Diagrama de Sequência — UC02

2.8.1 Linhas de Vida

```

:Praticante, :TimingDrillPage, :TimingDrillConfig, :TimingDrillSession,
:DrillTimer, :AudioEngine, :Announcer.

```

2.8.2 Mensagens (formato sintético)

```

Praticante -> TimingDrillPage : preencher_campos()
Praticante -> TimingDrillPage : click(btn-start)
TimingDrillPage -> TimingDrillConfig : new(duration, min, max, technique)
TimingDrillPage -> TimingDrillConfig : validate()

alt configuracao invalida
    TimingDrillConfig --> TimingDrillPage : ValueError
    TimingDrillPage -> TimingDrillPage : mostrar_erro()
else configuracao valida
    TimingDrillPage -> TimingDrillSession : new(config)
    TimingDrillPage -> TimingDrillSession : on_event(handle_event)
    TimingDrillPage -> DrillTimer : new(session, update_display, on_countdown, audio)
    TimingDrillPage -> DrillTimer : start()

    DrillTimer -> AudioEngine : play_countdown_tick_signal() // x3
    DrillTimer -> TimingDrillSession : start()
    DrillTimer -> AudioEngine : play_countdown_end_signal()

```



```

loop a cada 1s ate session_end
    DrillTimer -> TimingDrillSession : tick()

    opt pausada ou finalizada
        // sem efeito; sai do tick
    end

TimingDrillSession -> TimingDrillSession : _on_tick()

alt elapsed >= total_duration
    TimingDrillSession -> TimingDrillPage : emit(SESSION_END)
    TimingDrillPage -> AudioEngine : play_end_signal()
    TimingDrillPage -> Announcer : announce("Fim da sessao")
    TimingDrillPage -> TimingDrillPage : voltar_ao_formulario()
else elapsed >= next_stimulus_at
    TimingDrillSession -> TimingDrillPage : emit(STIMULUS, {technique})
    TimingDrillPage -> AudioEngine : play_start_signal()
    TimingDrillPage -> Announcer : announce(technique)
    TimingDrillPage -> TimingDrillPage : destacar_estimulo()
    TimingDrillSession -> TimingDrillSession : _schedule_next()
end
end
end

opt usuario solicita pausa
    Praticante -> TimingDrillPage : click(btn-pause)
    TimingDrillPage -> DrillTimer : pause()
    DrillTimer -> TimingDrillSession : pause()
end

opt usuario solicita parar
    Praticante -> TimingDrillPage : click(btn-stop)
    TimingDrillPage -> DrillTimer : stop()
    DrillTimer -> TimingDrillSession : stop()
end

```

2.8.3 Fragmento Combinado Obrigatório

O diagrama contém os seguintes fragmentos combinados (atende ao requisito de **ao menos um fragmento combinado**):

- `alt configuracao invalida / valida` — bifurca o fluxo principal pela validação.
- `loop a cada 1s ate session_end` — modela o ciclo de tick.
- `alt elapsed >= total_duration / elapsed >= next_stimulus_at` — distingue encerramento e emissão de estímulo.
- `opt pausada, opt parar` — fluxos alternativos do praticante.

2.9 Implementação

A implementação é fiel aos diagramas e está distribuída pelos seguintes arquivos:

- `src/domain/drill_config.py` — `TimingDrillConfig.validate`;
- `src/session/drill_session.py` — `DrillSession` (base);
- `src/session/timing_drill_session.py` — `TimingDrillSession` (`_schedule_next`, `_on_start`, `_on_tick`);
- `src/session/events.py` — `EventType`, `DrillEvent`;
- `src/ui/drill_timer.py` — `DrillTimer` (contagem regressiva e `setInterval`);
- `src/ui/pages/timing_drill_page.py` — `TimingDrillPage` (`render`, `mount`, `_on_start`, `_on_pause`, `_on_stop`, `_handle_event`);
- `src/audio/audio_engine.py` — `AudioEngine` (`play_start_signal`, `play_end_signal`, etc.);
- `src/audio/announcer.py` — `Announcer.announce`.

Capítulo 3

UC04 — Executar Footwork Drill (Vitor)

3.1 Descrição

Permite ao praticante configurar e executar um treino do tipo **Footwork Drill**, no qual o sistema anuncia por voz sintetizada movimentações **escolhidas aleatoriamente** da biblioteca, em intervalos aleatórios dentro de uma faixa configurada, durante uma duração total.

3.2 Fluxo Principal

1. O praticante acessa a tela do Footwork Drill.
2. O sistema verifica se há movimentações cadastradas (UC06):
 - se a biblioteca está vazia, exibe estado vazio e oferece atalho para “Gerenciar Movimentações”;
 - caso contrário, exibe o formulário com as movimentações disponíveis (caixas de seleção, todas marcadas por padrão), intervalo mínimo/máximo e duração total.
3. O praticante seleciona as movimentações desejadas e os parâmetros, acionando o início.
4. O sistema valida a configuração (RN: pelo menos uma movimentação selecionada, $\text{min} < \text{max}$, duração positiva).
5. O sistema oculta o formulário, exibe a tela de execução e inicia a contagem regressiva de 3 s (com beeps).
6. Ao término da contagem regressiva, o sistema inicia a sessão, **sorteia a primeira movimentação** e agenda o próximo instante.
7. A cada segundo, o sistema avança o contador e verifica:

- se a duração foi atingida, encerra (passo 9);
 - se o instante agendado foi atingido, sorteia a próxima movimentação (passo 8); caso contrário, prossegue para o próximo tick.
8. O sistema reproduz beep de início, anuncia o nome da movimentação por voz e atualiza o destaque visual; em seguida, sorteia o próximo instante.
 9. Ao fim da duração total, o sistema emite sinal sonoro de encerramento, anuncia mensagem de conclusão e retorna ao formulário.

3.3 Fluxos Alternativos

3.3.1 Pausa e Retomada

- A. O praticante aciona **Pausar**: o sistema suspende `tick` e nenhuma chamada é feita (RF10).
- B. Ao acionar **Retomar**, a contagem prossegue.

3.3.2 Nenhuma Movimentação Seleccionada

- 4a. O praticante desmarca todas as caixas e tenta iniciar.
- 4b. O sistema exibe mensagem de erro indicando a necessidade de ao menos uma movimentação seleccionada.
- 4c. Retorna ao passo 2.

3.3.3 Biblioteca Vazia

- 2a. A `FootworkMoveLibrary` está vazia.
- 2b. O sistema exibe a tela de estado vazio e oferece navegação para “Gerenciar Movimentações” (UC06).
- 2c. Fluxo encerra-se até que existam movimentações cadastradas.

3.3.4 Encerramento Antecipado

- O praticante aciona **Parar**; o sistema interrompe o `setInterval` e retorna ao formulário.

3.4 Pré-condições

A `FootworkMoveLibrary` deve conter ao menos uma movimentação (UC06).

3.5 Pós-condições

A sessão é concluída (ou encerrada antecipadamente) e a aplicação retorna ao estado de configuração.

3.6 Requisitos Relacionados

RF04, RF10, RF11, RN05, RN06.

3.7 Diagrama de Atividades — UC04

3.7.1 Texto Estruturado

[Start]

```
-> (Acessar pagina)
-> <D0: biblioteca vazia?>
    - sim -> (Exibir estado vazio + atalho UC06) -> [End]
    - nao ->
-> (Exibir formulario com movimentacoes)
-> (Praticante seleciona e preenche)
-> (Acionar inicio)
-> <D1: ao menos 1 movimento selecionado e parametros validos?>
    - nao -> (Exibir erro) -> (Exibir formulario)
    - sim ->
-> (Contagem regressiva 3..1)
-> (Iniciar sessao)
-> (Sortear primeira movimentacao + anunciar + beep)
-> (Agendar proximo instante)
-> (Avancar 1s) // ponto de retorno
-> <D2: elapsed >= duration?>
    - sim -> (Emitir end signal) -> (Anunciar fim) -> (Voltar ao formulario) -> [E]
    - nao ->
-> <D3: pausada?>
    - sim -> (Aguardar) -> (Avancar 1s)
    - nao ->
-> <D4: elapsed >= next_call_at?>
    - sim -> (Sortear movimento aleatorio)
        -> (Emitir MOVE_CALL: beep + anuncio + atualizar destaque)
        -> (Agendar proximo instante)
```

```
-> (Avancar 1s)
- nao -> (Avancar 1s)
```

3.7.2 Nós de Decisão (atende ao requisito mínimo)

- D0: existe ao menos uma movimentação cadastrada?
- D1: configuração e seleção válidas?
- D2: duração total atingida?
- D3: sessão pausada?
- D4: instante de chamada agendado atingido?

3.8 Diagrama de Sequência — UC04

3.8.1 Linhas de Vida

```
:Praticante, :FootworkDrillPage, :FootworkMoveLibrary, :FootworkDrillConfig,  
:FootworkDrillSession, :DrillTimer, :AudioEngine, :Announcer.
```

3.8.2 Mensagens

```
Praticante -> FootworkDrillPage : abrir_pagina()
FootworkDrillPage -> FootworkMoveLibrary : list_all()
FootworkMoveLibrary --> FootworkDrillPage : moves[]

alt biblioteca vazia
    FootworkDrillPage -> Praticante : exhibir_estado_vazio()
else
    FootworkDrillPage -> Praticante : exhibir_formulario(moves)
    Praticante -> FootworkDrillPage : selecionar_e_preencher()
    Praticante -> FootworkDrillPage : click(btn-start)
    FootworkDrillPage -> FootworkDrillConfig : new(selecionados, min, max, duration)
    FootworkDrillPage -> FootworkDrillConfig : validate()

    alt config invalida
        FootworkDrillConfig --> FootworkDrillPage : ValueError
        FootworkDrillPage -> Praticante : mostrar_erro()
    else config valida
        FootworkDrillPage -> FootworkDrillSession : new(config)
        FootworkDrillPage -> FootworkDrillSession : on_event(handle_event)
        FootworkDrillPage -> DrillTimer : new(session, update, countdown, audio)
```

```

FootworkDrillPage -> DrillTimer : start()

DrillTimer -> AudioEngine : play_countdown_tick_signal()    // x3
DrillTimer -> FootworkDrillSession : start()
FootworkDrillSession -> FootworkDrillSession : _call_move()
FootworkDrillSession -> FootworkDrillPage : emit(MOVE_CALL, {move})
FootworkDrillPage -> AudioEngine : play_start_signal()
FootworkDrillPage -> Announcer : announce(move.name)
FootworkDrillSession -> FootworkDrillSession : _schedule_next()
DrillTimer -> AudioEngine : play_countdown_end_signal()

loop a cada 1s ate SESSION_END
    DrillTimer -> FootworkDrillSession : tick()

    opt pausada ou finalizada
        // tick sem efeito
    end

    FootworkDrillSession -> FootworkDrillSession : _on_tick()

    alt elapsed >= total_duration
        FootworkDrillSession -> FootworkDrillPage : emit(SESSION_END)
        FootworkDrillPage -> AudioEngine : play_end_signal()
        FootworkDrillPage -> Announcer : announce("Fim da sessao")
        FootworkDrillPage -> Praticante : voltar_ao_formulario()
    else elapsed >= next_call_at
        FootworkDrillSession -> FootworkDrillSession : _call_move()
        FootworkDrillSession -> FootworkDrillPage : emit(MOVE_CALL, {move})
        FootworkDrillPage -> AudioEngine : play_start_signal()
        FootworkDrillPage -> Announcer : announce(move.name)
        FootworkDrillSession -> FootworkDrillSession : _schedule_next()
    end
end
end
end

opt usuario solicita pausa
    Praticante -> FootworkDrillPage : click(btn-pause)
    FootworkDrillPage -> DrillTimer : pause()

```

```

    DrillTimer -> FootworkDrillSession : pause()
end

opt usuario solicita parar
    Praticante -> FootworkDrillPage : click(btn-stop)
    FootworkDrillPage -> DrillTimer : stop()
    DrillTimer -> FootworkDrillSession : stop()
end

```

3.8.3 Fragmentos Combinados (atende ao requisito mínimo)

- alt biblioteca vazia / nao vazia;
- alt config invalida / valida;
- loop a cada 1s ate SESSION_END;
- alt elapsed >= total_duration / elapsed >= next_call_at;
- opt pausada, opt parar.

3.9 Implementação

- src/domain/footwork_move.py — FootworkMove;
- src/domain/libraries.py — FootworkMoveLibrary;
- src/domain/drill_config.py — FootworkDrillConfig.validate;
- src/session/drill_session.py — DrillSession;
- src/session/footwork_drill_session.py — FootworkDrillSession (_schedule_next, _call_move, _on_start, _on_tick);
- src/session/events.py — EventType.MOVE_CALL, DrillEvent;
- src/ui/drill_timer.py — DrillTimer;
- src/ui/pages/footwork_drill_page.py — FootworkDrillPage (render, mount, _on_start, _on_pause, _on_stop, _handle_event);
- src/audio/audio_engine.py — AudioEngine;
- src/audio/announcer.py — Announcer.

Capítulo 4

UC05 — Gerenciar Combos (Ruy)

4.1 Descrição

Permite ao praticante **listar**, **criar** e **excluir** combos na biblioteca de combos. Cada combo possui um nome (obrigatório e único na biblioteca) e uma descrição livre da sequência de técnicas. Toda alteração é persistida no `localStorage` por meio do `AppState` e do `StorageManager` (RNF08).

4.2 Fluxo Principal — Listar e Adicionar Combo

1. O praticante acessa a tela “Biblioteca de Combos”.
2. O sistema carrega os combos correntes via `AppState.combo_library.list_all()` e exibe a lista, juntamente com um formulário de cadastro.
3. O praticante preenche o nome do combo e (opcionalmente) a sequência de técnicas.
4. O praticante aciona **Adicionar**.
5. O sistema instancia `Combo(name, sequence)`. Em `__post_init__`, o nome é normalizado e validado (não pode ser vazio).
6. O sistema chama `combo_library.add(combo)`; se já existir combo de mesmo nome, é lançado `ValueError`.
7. O sistema chama `AppState.save()`, que serializa o estado e o persiste via `StorageManager`.
8. O sistema re-renderiza a página exibindo o novo combo na lista.

4.3 Fluxos Alternativos

4.3.1 Nome Inválido (passo 5)

- 5a. O nome está vazio.

- 5b. `Combo.__post_init__` levanta `ValueError("Combo name must not be empty.")`.
- 5c. O sistema exibe a mensagem de erro no formulário e mantém os campos preenchidos.
- 5d. Retorna ao passo 3.

4.3.2 Nome Duplicado (passo 6)

- 6a. Já existe um combo com o mesmo nome na biblioteca.
- 6b. `ComboLibrary.add` levanta `ValueError("A combo named '<name>' already exists.")`.
- 6c. O sistema exibe a mensagem de erro no formulário.
- 6d. Retorna ao passo 3.

4.3.3 Excluir Combo

1. O praticante aciona o botão **Excluir** no item desejado da lista.
2. O sistema chama `combo_library.remove(name)`.
3. O sistema chama `AppState.save()` para persistir.
4. O sistema re-renderiza a página, atualizando a lista exibida.

4.3.4 Estado Vazio

- Quando a `ComboLibrary` não possui combos, o sistema exibe a mensagem de “biblioteca vazia” em vez da lista de itens; o formulário de cadastro continua disponível.

4.4 Pré-condições

Nenhuma.

4.5 Pós-condições

A `ComboLibrary` reflete a alteração solicitada (inclusão ou exclusão) e o estado é persistido no `localStorage` (RNF08).

4.6 Requisitos Relacionados

RF06, RNF07, RNF08.

4.7 Diagrama de Atividades — UC05

4.7.1 Texto Estruturado

[Start]

```
-> (Acessar pagina Biblioteca de Combos)
-> (Carregar combos via combo_library.list_all)
-> (Renderizar lista + formulario)
-> <D0: acao do usuario?>
    - "adicionar" -> [add]
    - "excluir"   -> [del]
    - "sair"      -> [End]
```

[add]

```
-> (Preencher nome e sequencia)
-> (Click Adicionar)
-> <D1: nome vazio?>
    - sim -> (Exibir erro: nome vazio) -> (Renderizar lista + formulario)
    - nao ->
-> (Instanciar Combo(name, sequence))
-> <D2: nome ja existe na biblioteca?>
    - sim -> (Exibir erro: duplicado) -> (Renderizar lista + formulario)
    - nao ->
-> (combo_library.add)
-> (AppState.save)
-> (Re-renderizar lista) -> <D0>
```

[del]

```
-> (Click Excluir no item)
-> (combo_library.remove(name))
-> (AppState.save)
-> (Re-renderizar lista) -> <D0>
```

4.7.2 Nós de Decisão (atende ao requisito mínimo)

- D0: qual ação o usuário escolheu (adicionar, excluir, sair);
- D1: nome vazio?;
- D2: nome já existente na biblioteca?

4.8 Diagrama de Sequência — UC05

4.8.1 Linhas de Vida

:Praticante, :ComboLibraryPage, :AppState, :ComboLibrary, :Combo, :StorageManager, :StorageBackend.

4.8.2 Mensagens

```
Praticante -> ComboLibraryPage : abrir_pagina()
ComboLibraryPage -> AppState : combo_library
ComboLibraryPage -> ComboLibrary : list_all()
ComboLibraryPage -> Praticante : renderizar_lista + formulario

alt acao = Adicionar
  Praticante -> ComboLibraryPage : preencher(name, sequence)
  Praticante -> ComboLibraryPage : click(btn-add)
  ComboLibraryPage -> Combo : new(name, sequence)

  alt nome vazio
    Combo --> ComboLibraryPage : ValueError("Combo name must not be empty.")
    ComboLibraryPage -> Praticante : exibir_erro()
  else nome valido
    ComboLibraryPage -> ComboLibrary : add(combo)

    alt nome duplicado
      ComboLibrary --> ComboLibraryPage : ValueError("already exists")
      ComboLibraryPage -> Praticante : exibir_erro()
    else nome unico
      ComboLibrary -> ComboLibrary : _combos.append(combo)
      ComboLibraryPage -> AppState : save()
      AppState -> AppState : to_dict()
      AppState -> StorageManager : save(APP_STATE_KEY, dict)
      StorageManager -> StorageBackend : set_item(key, json)
      ComboLibraryPage -> ComboLibraryPage : rerender()
    end
  end
end

else acao = Excluir
  Praticante -> ComboLibraryPage : click(btn-delete, name)
  ComboLibraryPage -> ComboLibrary : remove(name)
```

```

    opt nome inexistente
      ComboLibrary --> ComboLibraryPage : KeyError
    end

    ComboLibraryPage -> AppState : save()
    AppState -> AppState : to_dict()
    AppState -> StorageManager : save(APP_STATE_KEY, dict)
    StorageManager -> StorageBackend : set_item(key, json)
    ComboLibraryPage -> ComboLibraryPage : rerender()
  end
end

```

4.8.3 Fragmentos Combinados (atende ao requisito mínimo)

- alt Adicionar / Excluir — bifurca o caso de uso pelas ações principais;
- alt nome vazio / nome valido — trata a validação do Combo;
- alt nome duplicado / nome unico — trata a unicidade na biblioteca;
- opt nome inexistente — caso defensivo na exclusão.

4.9 Implementação

- src/domain/combo.py — Combo (validação no `__post_init__`);
- src/domain/libraries.py — ComboLibrary (add, remove, get_by_name, update, list_all);
- src/persistence/app_state.py — AppState.save, AppState.load, AppState.to_dict;
- src/persistence/storage_manager.py — StorageManager.save, StorageManager.load;
- src/persistence/web_backend.py — adaptador para localStorage;
- src/ui/pages/combo_library_page.py — ComboLibraryPage (render, mount, _on_add, _on_delete, _bind_delete_buttons, _rerender);
- src/ui/page.py, src/ui/router.py, src/ui/app_context.py — infraestrutura comum.

Capítulo 5

Mapa de Implementação por Membro

Os códigos-fonte filtrados — isto é, apenas os arquivos diretamente relevantes a cada caso de uso — estão organizados em:

```
entrega3/  
  pedro_uc02_timing_drill/  
  vitor_uc04_footwork_drill/  
  ruy_uc05_gerenciar_combos/
```

Cada subpasta contém uma cópia dos arquivos Python utilizados no caso de uso correspondente, preservando a estrutura `src/...` para que o caminho de importação permaneça válido.

Capítulo 6

Rastreabilidade $UC \times RF \times RN \times$ Membro

UC	Requisitos	Regras	Membro
UC02	RF02, RF10, RF11	RN06	Pedro
UC04	RF04, RF10, RF11	RN05, RN06	Vitor
UC05	RF06, RNF07, RNF08	(RN nome único)	Ruy